

Flash Attention

目 录

1 引言：标准注意力的访存瓶颈	2
2 Flash Attention 核心：分块与在线 Softmax	2
2.1 分块读取 QKV	2
2.2 在线 Softmax	2
2.3 FP32 保存状态	3
3 因果注意力与滑动窗口	3
3.1 因果掩码的块级处理	3
3.2 滑动窗口跳过	3
4 GQA：避免重复 K/V	3
5 Prefill 与 Decode 的分块策略	4
5.1 Prefill：矩阵乘矩阵	4
5.2 Decode：矩阵乘向量	4
6 KV Cache 中的历史 token	4
7 实战：手算一次 Flash Attention	4
8 实现要求	5
9 本章你将学会	6
10 要点速查	7
11 小结	7

1 引言：标准注意力的访存瓶颈

标准自注意力的计算分三步：

$$A = QK^T, \quad P = \text{softmax}(A + M), \quad O = PV \quad (1.1)$$

其中 Q, K, V 均为 $L \times d$ 的矩阵， L 为序列长度， d 为头维度， M 为掩码矩阵。问题在于中间矩阵 A 和 P 的尺寸为 $L \times L$ ，当 L 较大时，它们占据大量 **Global Memory**（全局显存）。

标准实现的做法是：先计算完整的 $A = QK^T$ 写入显存，再读取 A 计算 softmax 写入 P ，最后读取 P 和 V 计算 O 。三个矩阵乘法之间，中间结果在片上寄存器和全局显存之间反复搬运，而全局显存的带宽远低于片上 SRAM。

直觉 想象你要统计全班 40 个同学两两之间的成绩差异。标准做法是：先把 $40 \times 40 = 1600$ 个差值全部算出来写到大白板上（全局显存），再从白板读取差值算 softmax 写回白板，最后再读白板算加权平均。白板很大但擦写很慢（HBM），你手上的草稿纸很小但擦写很快（SRAM）。Flash Attention 的思路是：不在白板上铺满 1600 个数，而是分块拿到草稿纸上算完，只把最终结果写回白板。

Flash Attention 通过分块计算和 **Online Softmax**（在线 softmax）算法，在不物化完整 $L \times L$ 注意力矩阵的前提下，精确地完成注意力计算，大幅减少全局显存的读写次数。

2 Flash Attention 核心：分块与在线 Softmax

2.1 分块读取 QKV

Flash Attention 不一次性加载整个 Q, K, V ，而是按块（tile）读取。外层循环遍历 KV 块，内层遍历 Q 块（或反过来，取决于实现策略）。每个块在片上 SRAM 中完成分数计算、掩码、softmax 更新和对 V 的加权累加，中间不写回全局显存。

2.2 在线 Softmax

标准 softmax 需要看到整行才能归一化： $P_i = \frac{e^{A_i - m}}{l}$ ，其中 m 是行最大值， l 是指数和。分块处理时，每处理完一个块只知道该块的局部信息，如何保证最终结果与标准 softmax 一致？

答案是在线 softmax。维护三个逐行的状态变量：

1. m ：当前已处理块的行最大值
2. l ：当前已处理块的指数和（未归一化）
3. o ：当前已处理块的未归一化输出

读入新的分数块 A_j 后，用以下公式更新三个状态：

$$m' = \max(m, \text{rowmax}(A_j)) \quad (2.1)$$

$$\alpha = e^{m - m'} \quad (2.2)$$

$$P_j = e^{A_j - m'} \quad (2.3)$$

$$l' = \alpha \cdot l + \text{rowsum}(P_j) \quad (2.4)$$

$$o' = \alpha \cdot o + P_j V_j \quad (2.5)$$

遍历完所有 KV 块后，最终输出为 $O = \frac{o}{l}$ 。

直觉 关键在于：当遇到一个新的块，其局部最大值 m_j 大于当前全局 m 时，之前累积的 l 和 o 都需要重新缩放。缩放因子 $\alpha = e^{m-m'}$ 正好补偿了最大值变化带来的偏移。更新后 m', l', o' 成为新的全局状态，可以继续处理下一个块。整个过程不需要保存任何中间矩阵。

2.3 FP32 保存状态

m, l 和累加器 o 必须用 FP32 保存。原因有二：一是 $e^{A_j-m'}$ 的指数运算在 FP16 下可能溢出或下溢；二是多块累加时 FP16 的精度不足以保证收敛。输入 QKV 可以是 BF16，但状态变量的计算和中间累加在 FP32 中进行。

3 因果注意力与滑动窗口

3.1 因果掩码的块级处理

因果注意力要求位置 i 的 query 只关注位置 0 到 i 的 key。在块级别上，这分为三种情况：

1. KV 块在 Q 块之前 ($i_{kv} < i_q$)：该 KV 块的所有位置都在因果范围内，无需掩码，全块计算
2. KV 块在 Q 块之后 ($i_{kv} > i_q$)：该 KV 块的所有位置都在因果范围外，直接跳过，不计算
3. KV 块与 Q 块重叠 ($i_{kv} = i_q$)：块内需要逐元素施加因果掩码，上三角部分填 $-\infty$

第三种情况是性能关键：只需要在一个块内做精细掩码，其余块要么全算要么跳过。这使得因果注意力的计算量约为无因果版本的一半，而不是在完整 $L \times L$ 矩阵上先计算再掩码。

3.2 滑动窗口跳过

滑动窗口注意力要求 query 只关注最近 w 个位置的 key。在块级别上，窗口外的 KV 块直接跳过，不计算也不加载。这比先计算再用 $-\infty$ 掩码高效得多，因为跳过的块根本不产生访存和计算开销。

Gemma4-12B 中滑动窗口层 $w = 1024$ 。如果 block size 为 64，窗口内只有 $\frac{1024}{64} = 16$ 个块需要计算，其余全部跳过。长序列时这能省去绝大部分计算。

4 GQA：避免重复 K/V

Grouped Query Attention（分组查询注意力）让多个 query head 共享同一组 KV head。Gemma4-12B 的滑动窗口层有 16 个 query head 和 8 个 KV head，每 2 个 query head 共享 1 个 KV head；全局层有 16 个 query head 和 1 个 KV head，所有 query head 共享同一组 KV。

朴素做法是用 repeat_kv 把 K 和 V 沿 head 维度复制到与 Q 相同的数量，再执行标准注意力。这会物化重复的 K/V，浪费显存和带宽。

直觉 repeat_kv 就像给 16 个学生每人复印一份相同的参考书。参考书内容完全一样，何必复印 16 份？不如让每 2 个学生共用一本书，只是读的时候注意翻到正确的页。

Flash Attention 的做法是在 kernel 内部建立 query head 到 KV head 的映射：query head h 使用 KV head $h \div g$ ，其中 $g = \frac{H_q}{H_{kv}}$ 是共享比例。这样 K 和 V 只需存储一份，kernel 计算时按映射关系读取，避免物化重复数据。

5 Prefill 与 Decode 的分块策略

5.1 Prefill：矩阵乘矩阵

Prefill 阶段输入一个长度为 $q_{len} > 1$ 的 prompt，Q 是一个矩阵。注意力计算是矩阵乘矩阵，GPU 并行度高，适合用较大的 tile 在 Tensor Core 上高效计算。分块策略侧重于提高 M 维（query 行数）的 tile size，充分利用 Tensor Core 的计算能力。

5.2 Decode：矩阵乘向量

Decode 阶段每次只生成一个 token， $q_{len} = 1$ ，Q 退化为一个向量。注意力计算变成矩阵乘向量，算术强度极低，受权重读取和算子发射开销限制。此时分块策略侧重于减少 K/V 的重复读取，而非提高 Tensor Core 利用率。

阶段	q_{len}	瓶颈
Prefill	> 1	计算密集，Tensor Core 利用率高
Decode	$= 1$	访存密集，算子发射开销大

同一个 kernel 在 prefill 和 decode 下的最优 tile 配置不同。工业级引擎通常为 prefill 和 decode 分别编写或选择不同的 kernel。Lab5 中可以用同一套代码但不同 tile 参数来适配。

6 KV Cache 中的历史 token

Decode 阶段，KV Cache 中已经存放了之前所有 token 的 K 和 V。新 token 的注意力需要遍历整个 KV Cache。当 KV Cache 的长度不是 block size 的整数倍时，最后一个 tile 不完整，越界位置需要掩码为 $-\infty$ ，避免参与 softmax。

具体做法是：计算最后一个 tile 的分数后，对越界位置施加掩码。如果 block size 为 B ，KV Cache 实际长度为 S ，最后一个完整 tile 后剩余 $S \bmod B$ 个有效位置，其余 $B - S \bmod B$ 个位置掩码。

7 实战：手算一次 Flash Attention

我们用极小的数值手动走一遍在线 softmax 的完整流程，验证它与标准 softmax 结果一致。

例 取 $q_{len} = 1, L = 4, d = 1$ 。设：

$$Q = (1), \quad K = (1, 0, 1, 0), \quad V = (1, 2, 3, 4) \quad (7.1)$$

即 query 是标量 1，K 有 4 个标量，V 有 4 个标量。

标准注意力： $A = QK^T = (1 \times 1, 1 \times 0, 1 \times 1, 1 \times 0) = (1, 0, 1, 0)$ 。

$$m = \max(1, 0, 1, 0) = 1 \quad (7.2)$$

$$e^{A-m} = (e^0, e^{-1}, e^0, e^{-1}) = (1, 0.368, 1, 0.368) \quad (7.3)$$

$$l = 1 + 0.368 + 1 + 0.368 = 2.736 \quad (7.4)$$

$$P = \frac{1, 0.368, 1, 0.368}{2.736} = (0.3655, 0.1345, 0.3655, 0.1345) \quad (7.5)$$

$$O = PV = 0.3655 \times 1 + 0.1345 \times 2 + 0.3655 \times 3 + 0.1345 \times 4 \quad (7.6)$$

$$= 0.3655 + 0.2691 + 1.0965 + 0.5381 = 2.2692 \quad (7.7)$$

Flash Attention (分 2 块, 每块 2 个元素):

块 0 ($K_0 = (1, 0), V_0 = (1, 2)$):

$$A_0 = (1 \times 1, 1 \times 0) = (1, 0) \quad (7.8)$$

$$m = \max(1, 0) = 1 \quad (7.9)$$

$$P_0 = (e^{1-1}, e^{0-1}) = (1, 0.368) \quad (7.10)$$

$$l = 1 + 0.368 = 1.368 \quad (7.11)$$

$$o = 1 \times 1 + 0.368 \times 2 = 1 + 0.736 = 1.736 \quad (7.12)$$

块 1 ($K_1 = (1, 0), V_1 = (3, 4)$):

$$A_1 = (1 \times 1, 1 \times 0) = (1, 0) \quad (7.13)$$

$$m' = \max(1, 1) = 1 \quad (7.14)$$

$$, \alpha = e^{1-1} = 1$$

$$P_1 = (e^{1-1}, e^{0-1}) = (1, 0.368) \quad (7.15)$$

$$l' = 1 \times 1.368 + 1 + 0.368 = 2.736 \quad (7.16)$$

$$o' = 1 \times 1.736 + 1 \times 3 + 0.368 \times 4 = 1.736 + 3 + 1.472 = 6.208 \quad (7.17)$$

最终输出: $O = \frac{o'}{l'} = \frac{6.208}{2.736} = 2.2690$

与标准注意力的 2.2692 基本一致 (差异来自小数舍入)。注意整个过程中我们从未物化 4×4 的注意力矩阵, 只维护了 m, l, o 三个标量。

直觉 这个例子揭示了 Flash Attention 的精髓: 分块处理时, 每块只需记录局部最大值 m 和局部和 l , 合并时用新旧最大值的差做一次 rescale。不需要保存整个注意力矩阵, 中间状态只有三个标量。这就是“以算换存”的精确含义。

8 实现要求

Lab5 要求用 **Triton** 或 **TileLang** 编写 Flash Attention kernel, 不得调用 FlashAttention、xFormers、PyTorch SDPA (F.scaled_dot_product_attention) 等现成库。

核心实现要点:

1. 用 Triton 的 `tl.load` 分块加载 Q、K、V, `tl.store` 写回结果
2. 在寄存器中计算分数 $A_j = Q_i K_j^T$, 施加因果或滑动窗口掩码
3. 用 FP32 维护 m, l, o , 按在线 softmax 公式更新

4. 遍历完所有 KV 块后，输出 $O = \frac{o}{l}$
5. GQA 映射在 kernel 内部完成，不调用 repeat_kv
6. 滑动窗口和因果掩码的块级跳过逻辑

```
@triton.jit
def flash_attn_kernel(
    Q_ptr, K_ptr, V_ptr, O_ptr,
    seq_len, scale,
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    ...
):
    m = tl.full([BLOCK_M], -float("inf"), dtype=tl.float32)
    l = tl.zeros([BLOCK_M], dtype=tl.float32)
    acc = tl.zeros([BLOCK_M, D], dtype=tl.float32)

    for j in range(0, seq_len, BLOCK_N):
        k = tl.load(K_ptr + offsets_n)
        v = tl.load(V_ptr + offsets_n)
        a = tl.dot(q, k.T) * scale
        a = apply_mask(a, ...)
        m_new = tl.maximum(m, tl.max(a, axis=1))
        alpha = tl.exp(m - m_new)
        p = tl.exp(a - m_new[:, None])
        l = alpha * l + tl.sum(p, axis=1)
        acc = alpha[:, None] * acc + tl.dot(p.to(v.dtype), v)
        m = m_new

    o = acc / l[:, None]
    tl.store(O_ptr + offsets, o)
```

逐行说明：m、l、acc 分别初始化为 $-\infty$ 、0 和零矩阵。循环内先加载 K、V 块，计算分数 a，施加掩码。然后更新最大值 m_new，计算缩放因子 alpha 和指数 p，更新 l 和 acc。循环结束后除以 l 得到最终输出。整个过程在寄存器中完成，不写回中间结果。

9 本章你将学会

1. 解释标准注意力物化 $L \times L$ 矩阵导致的访存瓶颈
2. 描述 Flash Attention 的分块读取策略和在线 softmax 算法
3. 手动推导 m, l, o 三状态的更新公式，并用小数值验证
4. 实现因果掩码的块级跳过（全算、全跳、块内掩码三种情况）
5. 实现滑动窗口的块级跳过
6. 在 kernel 内部处理 GQA 映射，避免 repeat_kv
7. 区分 prefill 和 decode 的分块策略选择
8. 处理 KV Cache 最后一个不完整 tile 的越界掩码
9. 用 Triton 或 TileLang 编写 Flash Attention kernel

10 要点速查

要点	说明
标准注意力中间矩阵	A 和 P 为 $L \times L$, 反复读写 global memory
Flash Attention	分块计算, 不物化完整注意力矩阵
在线 softmax 三状态	m (行最大值), l (指数和), o (未归一化输出)
状态更新	$m' = \max(m, \text{rowmax}(A_j))$, $\alpha = e^{m-m'}$
最终输出	$O = \frac{o}{l}$
FP32 状态	m, l, o 必须用 FP32 避免溢出
因果掩码块级	之前的块全算, 之后的块跳过, 重叠块内掩码
滑动窗口跳过	窗口外块直接跳过, 不计算不加载
GQA 映射	query head h 用 KV head $h \div g$, 避免 repeat
Prefill 分块	大 M tile, Tensor Core 高利用
Decode 分块	$q_{\text{len}} = 1$, 减少 K/V 重复读取
不完整 tile	越界位置掩码为 $-\infty$

11 小结

Flash Attention 是精确注意力计算的 I/O 优化版本。它不改变注意力的数学结果, 而是通过分块读取 QKV 和在线 softmax 算法, 避免了 $L \times L$ 中间矩阵的物化。在线 softmax 的核心是维护三个状态变量 m, l 和 o , 每处理一个新块时用新旧最大值的差做一次 rescale, 遍历完成后除以 l 得到最终输出。

在 Gemma4-12B 的场景中, 因果掩码和滑动窗口都可以用块级跳过高效处理: 因果注意力只遍历因果范围内的 KV 块, 滑动窗口只遍历窗口内的 KV 块。GQA 映射在 kernel 内部完成, 避免物化重复 K/V。Prefill 和 decode 两种阶段需要不同的分块策略: prefill 侧重 Tensor Core 利用率, decode 侧重减少访存。Lab5 要求用 Triton 或 TileLang 从零实现, 不依赖现成库, 这是深入理解注意力计算和 GPU 编程的关键一步。

讲义基于 HPC Lab5 实验指导编写